Design and Implementation of a Custom Network Proxy Server

Dara Zenas Zephaniah
23321010

Enjarapu Premchand
23321013

Abstract

This report details the architectural design and implementation of a custom HTTP Forward Proxy Server engineered in Python. The system utilizes a fixed-size thread pool concurrency model to maintain a stable resource envelope while serving multiple clients. To address modern web workflows, the proxy integrates transparent HTTPS CONNECT tunneling using single-threaded bidirectional I/O multiplexing via non-blocking socket polling. Furthermore, the implementation features a rule-based recursive domain access control filter and an in-memory, thread-safe Least Recently Used (LRU) cache constructed from scratch with a custom doubly linked list and hash map. Complete configuration mechanics, dataflow models, operational logic, and automated testing architectures are discussed.

Contents

1 Introduction and Problem Specification

A network proxy server acts as an intermediary software engine situated between client agents and external destination hosts. In modern network architectures, proxies provide strategic interception points to enforce access policies, reduce duplicate wide-area network traffic through caching, and audit transactional access.

The core objective of this project is to construct a production-ready, highly stable forward proxy server over TCP utilizing low-level socket programming. The target implementation must satisfy several constraints:

- Establish robust TCP socket listeners and manage client-server handshakes.
- Achieve system concurrency utilizing a bounded resource management pattern.
- Implement rigorous HTTP request line, target URI, and header parsing.
- Support both relative and absolute proxy target formats.
- Stream connections directly to prevent large memory overheads during wide-area network transfers.
- Mitigate systemic failures through graceful socket and signal lifecycle management.

Beyond basic HTTP forwarding, this design implements two significant extensions:

1. **HTTPS CONNECT Tunneling**: Establishing secure, encrypted end-to-end TLS conduits utilizing non-blocking sockets and bidirectional `select.select()` multiplexing within single worker threads.
2. **Thread-Safe LRU Cache**: Reducing target latency through an in-memory caching engine using a custom-built doubly linked list and hash map, synchronized with mutual exclusion locks to prevent race conditions during parallel client requests.

—

2 Project Directory Layout

The codebase adheres to a strict modular separation of concerns to maximize unit-testing capabilities and prevent low-level networking components from overlapping with business, routing, or filtering rules. This layout uses pure ASCII branches for compile safety.

```
custom-network-proxy-server/
|-- config/
|   |-- blocked_domains.txt     # Domain policy block rules
|   +-- server_config.json      # JSON server configurations
|-- docs/
|   |-- README.md               # Quickstart guide
|   +-- architecture.md         # Markdown design manual
|-- logs/
|   +-- proxy.log               # Live persistent system logs
|-- src/
|   |-- proxy.py                # Listening socket loop & worker threads
|   |-- parser.py               # Robust raw HTTP parser
|   |-- filter.py               # Recursive suffix filter
|   |-- cache.py                # Custom thread-safe LRU Cache
|   +-- logger.py               # Synchronous dual-sink logger
+-- tests/
    +-- test_proxy.py           # Automated integration test suite
```

Listing 1: Repository File Structure

—

3 Detailed Component Design

3.1 Connection Core (src/proxy.py)

The connection core serves as the system coordinator. It instantiates the listening server socket bound to a configurable address and port. It registers OS signal handlers for clean resource teardown and manages the main client accept loop. Upon accepting a new TCP handshake, it hands off the socket descriptor directly to a thread pool for processing.

3.2 HTTP Parser (src/parser.py)

The parser handles string parsing of raw HTTP streams. It implements a header accumulation loop that reads data in small fragments (e.g., 1024 bytes) until the sequence `\r\n\r\n` is found. Once accumulated, it parses:

- **Request Line**: HTTP Method, Target URI, and HTTP Version.
- **Target Resolution**: Resolves absolute URLs (e.g., `http://example.com/index.html`) and relative URLs (utilizing the parsed `Host` header) to extract the canonical host and port.
- **Payload Bodies**: Reads accompanying body bytes according to the parsed `Content-Length` value.

3.3 Recursive Suffix Filter (src/filter.py)

The policy filter evaluates target domains against the system blocklist. It converts domains to lowercase and strips extraneous spaces. To handle subdomains elegantly, it splits the request host and recursively checks parent suffixes. For instance, if `blockedexample.com` is blacklisted, a request to `sub.blockedexample.com` is blocked automatically.

3.4 Thread-Safe LRU Cache (src/cache.py)

The cache implements an in-memory Least Recently Used (LRU) key-value store. It is built from scratch without standard library wrappers:

- **Hash Map**: Provides $O(1)$ node lookup times using the normalized request path as the key.
- **Doubly Linked List**: Maintains access order. Nodes at the head represent Most Recently Used (MRU) items, while nodes at the tail represent Least Recently Used (LRU) items.
- **Mutex Lock**: A mutual exclusion lock (`threading.Lock`) serializes pointer modifications to ensure safety when multiple worker threads read or write to the cache.

—

4 Concurrency Model and Resource Management

Three concurrency paradigms were evaluated for this implementation:

1. **Thread-per-Connection**: Spawns a dedicated thread for every incoming socket. While easy to write, this approach is vulnerable to thread exhaustion and significant performance drops under concurrent connection spikes.

2. **Event-Driven / Asynchronous Loop**: Scalable, but adds high complexity when integrating synchronous file-system writes (for logs) and managing in-memory cache coherence across parallel requests.

3. **Thread Pool (Selected)**: A thread pool utilizing a fixed count of worker threads is constructed during boot. This model limits the system's resource footprint, queueing excess connections during spikes and ensuring that server performance remains predictable under load.

Metric	Thread-per-Connection	Fixed Thread Pool
Resource Scaling	Unbounded ($O(N)$)	Bounded ($O(1)$ limit)
Overhead under Load	High Context Switching	Stable Queue Execution
Vulnerability	Thread Exhaustion / Denial of Service	Safe Resource Bounds

Table 1: Concurrency Model Comparison

—

5 System Data Flow and Operational Workflows

5.1 Standard HTTP GET Workflow

When a client requests an unencrypted webpage via the proxy:

1. The client connects to the proxy port. The main thread accepts the connection and dispatches the task to the next available worker thread.

2. The worker thread reads and parses the HTTP request headers.

3. The worker queries the `URLFilter`. If the host is blocked, it responds immediately with an `HTTP/1.1 403 Forbidden` payload and closes the connection.

4. The worker checks the `LRUCache`. If a cache hit occurs, the cached data is streamed directly to the client, and the transaction is complete.

5. On a cache miss, the worker establishes a TCP socket to the remote destination server.

6. The proxy reconstructs the HTTP request line (with relative target mapping) and forwards it to the target.

7. The remote response is received, streamed immediately to the client in small chunks (preventing memory buffering issues), and saved to the LRU Cache.

5.2 HTTPS CONNECT Tunneling Workflow

Secure HTTPS connections utilize the `CONNECT` request method. Since the proxy cannot inspect or decrypt secure payloads, it switches to a raw bidirectional tunnel:

1. The client sends a `CONNECT target:443 HTTP/1.1` request.

2. If the target is not blocked, the proxy establishes a raw TCP connection to the destination host on port 443.

3. The proxy returns an unencrypted `HTTP/1.1 200 Connection Established` response to the client.

4. Both sockets are set to non-blocking mode.
5. The worker thread enters a polling loop using `select.select()`, allowing the operating system to wake the thread when data is ready to be read from either socket.
6. Encrypted data is piped transparently between the client and the destination server until one of the connections is closed.

—

6 System Configurations and Log Structures

6.1 System Settings (config/server_config.json)

The server configuration parameters are specified in a structured JSON schema:

```
{
  "host": "127.0.0.1",
  "port": 8888,
  "thread_pool_size": 16,
  "log_file": "logs/proxy.log",
  "blocked_domains_file": "config/blocked_domains.txt",
  "timeout": 15,
  "cache_capacity": 5
}
```

Listing 2: Proxy Server JSON Configuration

6.2 Domain Access Rules (config/blocked_domains.txt)

Rules are defined as simple, lowercase host suffix lines:

```
# Restricted domains list - one domain per line
blockedexample.com
badsite.org
unsafe-domain.net
```

Listing 3: Blocked Domains Configuration File

—

7 Verification, Automated Testing, and System Logs

7.1 Automated Testing Suite

An integration suite implemented in `tests/test_proxy.py` validates all system components. The tests use Python's `subprocess` module to run automated `curl` requests through the proxy, verifying standard forwarding, caching, HTTPS tunneling, and domain-blocking rules:

- **Test 1 (HTTP Forwarding/Cache)**: Verifies that standard HTTP GET requests succeed with an HTTP 200 code. A second call checks if the resource is served directly from the LRU Cache.
- **Test 2 (HTTPS Tunneling)**: Establishes a raw TCP tunnel, completes a TLS handshake, and validates the HTTP response.
- **Test 3 (HTTP Policy Blocking)**: Checks if HTTP requests to blocked hosts are intercepted and returned with an HTTP 403 Forbidden code.
- **Test 4 (HTTPS CONNECT Blocking)**: Confirms that attempts to establish secure tunnels to blocked targets are aborted during the tunnel setup phase, returning an HTTP 403 Forbidden code.

7.2 Standard System Logs

The proxy's logging output records each connection's lifecycle, the handling worker thread, policy decisions, and caching behavior:

```
[2026-06-04 20:35:43] [INFO] [MainThread] Proxy server successfully started on 127.0.0.1:8888
[2026-06-04 20:35:51] [INFO] [ProxyWorker_0] Handling connection from 127.0.0.1:50993
[2026-06-04 20:35:51] [INFO] [ProxyWorker_0] Parsed Request -> HEAD example.com:80/
[2026-06-04 20:35:51] [INFO] [ProxyWorker_0] Success -> HEAD example.com | Transferred: 269 bytes
[2026-06-04 20:35:58] [INFO] [ProxyWorker_0] Handling connection from 127.0.0.1:50995
[2026-06-04 20:35:58] [INFO] [ProxyWorker_0] Parsed Request -> GET example.com:80/
[2026-06-04 20:35:58] [INFO] [ProxyWorker_0] Cache STORE -> Saved 'example.com/' to LRU Cache.
[2026-06-04 20:36:00] [INFO] [ProxyWorker_0] Handling connection from 127.0.0.1:50997
[2026-06-04 20:36:00] [INFO] [ProxyWorker_0] Parsed Request -> CONNECT example.com:443/
[2026-06-04 20:36:00] [INFO] [ProxyWorker_0] Establishing HTTPS Tunnel to example.com:443
[2026-06-04 20:36:17] [WARNING] [ProxyWorker_0] Blocked Request -> Host 'blockedexample.com' matched policies in blocklist.
```

Listing 4: System Logging Output

—

8 Security Considerations and Operational Constraints

- **Zero TLS Inspection**: To preserve privacy, the proxy does not inspect secure payloads. Encrypted traffic passes through the proxy without decrypting, leaving it uninspected.
- **Robust String Processing**: Target hosts are converted to lowercase and leading/trailing whitespace is stripped, preventing evasion techniques like mixing case (e.g., `BlOcKeDeXaMpLe.com`).
- **Persistent Connection Optimization**: To simplify connection state tracking and prevent socket leaks, the proxy forces the header `Connection: close` on non-tunnel requests.
- **Socket Error Resilience**: Every phase of request forwarding is wrapped in exception blocks. Sockets are closed in a `finally` block to prevent resource leaks during transmission failures.